

C# 中的 Google Cloud Functions

來源：<https://codelabs.developers.google.com/codelabs/cloud-functions-csharp?hl=zh-tw>

步驟數：9

在本程式碼研究室中，您將瞭解 C# 語言的 Google Cloud Run 函式。具體來說，您將部署 C# 函式來回應各種 Google Cloud 來源的 HTTP 和 CloudEvents。

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 事前準備](#)
4. [4. .NET 適用的 Functions Framework](#)
5. [5. HTTP 函式](#)
6. [6. CloudEvent 函式 - GCS](#)
7. [7. CloudEvent 函式 - Pub/Sub](#)
8. [8. CloudEvent 函式 - 無型別](#)
9. [9. 恭喜！](#)

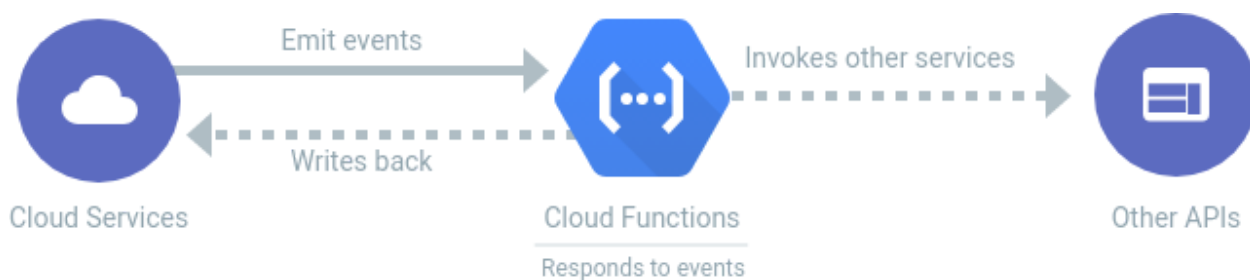
步驟 1 · 約 0 分鐘

1. 簡介

[Google Cloud Run 函式](#)是事件導向的無伺服器運算平台，您可以使用 Cloud Run functions 編寫程式碼，不必擔心資源佈建或擴充作業，以因應不斷變化的需求。

Cloud Run 函式分為兩種類型：

- HTTP 函式會回應 HTTP 要求。
- 事件函式是由事件觸發，例如訊息發布至 [Cloud Pub/Sub](#) 或檔案上傳至 [Cloud Storage](#)。



本程式碼研究室將逐步說明如何使用 C# 建立自己的 Cloud Run 函式。具體來說，您會部署 C# 函式，回應來自各種 Google Cloud 來源的 HTTP 和 CloudEvents。

課程內容

- 適用於 .NET 的 Functions Framework。
 - 如何編寫 HTTP 函式。
 - 如何編寫回應 Cloud Storage 事件的事件觸發函式。
 - 如何編寫回應 Cloud Pub/Sub 事件的事件觸發函式。
 - 如何編寫會回應任何類型事件的事件觸發函式。
-

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The screenshot shows the Google Cloud Platform interface. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' button, which is circled in blue. Below this, there's a 'Select a project' section with a search bar. Further down, there's a 'New Project' section. In the 'New Project' section, the 'Project name' field is filled with 'my-kubernetes-codelab' and is circled in blue. Below the name field, the 'Project ID' is shown as 'my-kubernetes-codelab' with a note that it cannot be changed later. The 'Location' is set to 'No organization'. At the bottom of the 'New Project' section, there are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新該位置資訊。
- **專案 ID** 在所有 Google Cloud 專案中不得重複，且設定後即無法變更。Cloud 控制台會自動產生不重複的字串，通常您不需要在意這個字串。在大多數程式碼研究室中，您需要參照專案 ID (通常會標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試自訂名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間都會維持這個設定。
- 請注意，部分 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中不得重複，選定後即為專屬 ID，其他使用者無法使用。您是該 ID 的唯一使用者。即使刪除專案，該 ID 也無法再使用

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成本程式碼研究室的費用應該不高，甚至完全免費。如要關閉資源，避免產生本教學課程以外的費用，您可以刪除自己建立的資源，或刪除整個專案。Google

Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

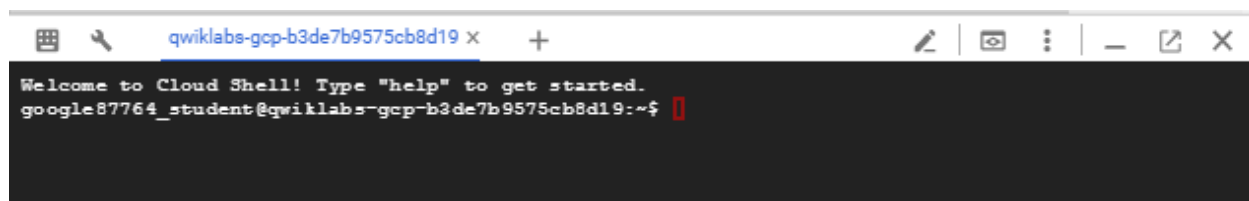
啟動 Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

在 [Google Cloud 控制台](#) 中，點選右上工具列的 Cloud Shell 圖示：



佈建並連線至環境的作業需要一些時間才能完成。完成後，您應該會看到如下的內容：

A screenshot of a web browser window showing the Google Cloud Shell terminal. The browser's address bar displays 'qwiklabs-gcp-b3de7b9575cb8d19'. The terminal window has a dark background and shows the following text: 'Welcome to Cloud Shell! Type "help" to get started.' followed by the prompt 'google87764_student@qwiklabs-gcp-b3de7b9575cb8d19:~\$' and a red cursor. The browser window includes standard navigation buttons and a tab titled 'qwiklabs-gcp-b3de7b9575cb8d19 x'.

這部虛擬機器搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。您可以在瀏覽器中完成本程式碼研究室的所有作業。您不需要安裝任何軟體。

步驟 3 · 約 0 分鐘

3. 事前準備

在 Cloud Shell 中執行下列指令，啟用必要服務：

```
gcloud services enable \  
  artifactregistry.googleapis.com \  
  cloudbuild.googleapis.com \  
  cloudfunctions.googleapis.com \  
  eventarc.googleapis.com \  
  run.googleapis.com
```

接著設定地區。

```
REGION=<YOUR_REGION>
```

在本程式碼研究室中，您將建立具備必要 EventArc 權限和 Cloud Run 叫用者角色的服務帳戶，以便接收 Cloud Storage 的事件並叫用 Cloud Run 函式。

首先，請建立服務帳戶。

```
PROJECT_ID=$(gcloud config get-value core/project)  
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --  
format='value(projectNumber)')  
  
SERVICE_ACCOUNT="cloud-run-functions"  
SERVICE_ACCOUNT_ADDRESS=$SERVICE_ACCOUNT@$PROJECT_ID.iam.gserviceaccount.com  
  
gcloud iam service-accounts create $SERVICE_ACCOUNT \  
  --display-name="Cloud Run functions Eventarc service account"
```

接著，將專案的 Eventarc 事件接收者角色 (roles/eventarc.eventReceiver) 授予與 Eventarc 觸發條件相關聯的服務帳戶，讓觸發條件可以接收事件提供者的事件。

```
gcloud projects add-iam-policy-binding $PROJECT_ID \  
  --member serviceAccount:$SERVICE_ACCOUNT_ADDRESS \  
  --role=roles/eventarc.eventReceiver
```

接著，將 Cloud Run 叫用者角色授予服務帳戶，讓該帳戶可以叫用函式。

```
gcloud projects add-iam-policy-binding $PROJECT_ID \  
  --member serviceAccount:$SERVICE_ACCOUNT_ADDRESS \  
  --role=roles/run.invoker
```

4. .NET 適用的 Functions Framework

[.NET 適用的 Functions Framework](#) 是開放原始碼的 FaaS (函式即服務) 架構，可供您編寫可攜式 .NET 函式，由 Google Cloud Functions 團隊提供。

Functions Framework 可讓您編寫簡易函式，並在許多不同環境中執行，包括：

- Google Cloud Run functions
- 您的本機開發機器
- Cloud Run 和 Cloud Run on GKE
- 以 Knative 為基礎的環境

在本程式碼研究室中，您將使用 .NET 適用的 Functions Framework 及其範本，以 C# 建立及部署 Cloud Functions。

在 Cloud Shell 中執行下列指令，安裝 `dotnet` 的 Cloud Functions 範本：

```
dotnet new install Google.Cloud.Functions.Templates
```

這會安裝 3 個 `dotnet` 的範本。每個範本都有 C#、F# 和 VB 版本 (但您只會在實驗室中使用 C#)。您可以執行下列指令，確認範本是否已安裝：

```
dotnet new list
```

Templates	Short Name
Google Cloud Functions CloudEvent Function	gcf-event
Google Cloud Functions CloudEvent Function (Untyped)	gcf-untyped-event
Google Cloud Functions HttpFunction	gcf-http

步驟 5 · 約 4 分鐘

5. HTTP 函式

您將建立及部署 HTTP 函式，用來回應 HTTP 要求。

使用 `gcf-http` 範本建立 HTTP 函式：

```
mkdir HelloHttp
cd HelloHttp
dotnet new gcf-http
```

這個指令會建立專案和 `Function.cs` 檔案，用來回應 HTTP 要求。

在 `.csproj` 檔案中，將目標架構變更為 `net8.0`：

```
<TargetFramework>net8.0</TargetFramework>
```

如要將 Cloud Run 函式直接部署至 Cloud Run，請執行下列指令：

```
gcloud beta run deploy hello-http-function \
  --source . \
  --function HelloHttp.Function \
  --base-image dotnet8 \
  --region $REGION \
  --allow-unauthenticated
```

如要部署為 Cloud Functions 第 2 代，請使用下列指令：

```
gcloud functions deploy hello-http-function \
  --allow-unauthenticated \
  --entry-point HelloHttp.Function \
  --gen2 \
  --region $REGION \
  --runtime dotnet8 \
  --trigger-http
```

部署函式後，您可以使用下列 `curl` 指令叫用函式：

```
SERVICE_URL=$(gcloud run services describe hello-http-function --platform managed --
region $REGION --format 'value(status.url)')

curl $SERVICE_URL
```


步驟 6 · 約 4 分鐘

6. CloudEvent 函式 - GCS

您將建立及部署 CloudEvent 函式，以回應 Google Cloud Storage (GCS) 事件。

首先，請建立 Cloud Storage bucket。這是您稍後要監聽事件的 bucket：

```
BUCKET_NAME="cloud-functions-bucket-${PROJECT_ID}"
gsutil mb -l us-central1 gs://${BUCKET_NAME}
```

使用 `gcf-event` 範本建立 CloudEvent 函式：

```
cd ..
mkdir HelloGcs
cd HelloGcs
dotnet new gcf-event
```

這會建立專案和 `Function.cs` 檔案，以回應 `CloudEvent` 要求。此外，也會將 `CloudEvent` 的資料剖析為 `StorageObjectData`。

在 `.csproj` 檔案中，將目標架構變更為 `net8.0`：

```
<TargetFramework>net8.0</TargetFramework>
```

如果要將 Cloud Run 函式直接部署至 Cloud Run，請先部署函式，然後為函式建立觸發條件。

```
gcloud beta run deploy hello-gcs-function \
  --source . \
  --function HelloGcs.Function \
  --region $REGION \
  --base-image dotnet8 \
  --no-allow-unauthenticated
```

現在為 Cloud Run 函式建立觸發條件

```
BUCKET_REGION=$REGION

gcloud eventarc triggers create hello-gcs-function-trigger \
  --location=$REGION \
  --destination-run-service=hello-gcs-function \
  --destination-run-region=$BUCKET_REGION \
  --event-filters="type=google.cloud.storage.object.v1.finalized" \
  --event-filters="bucket=$BUCKET_NAME" \
  --service-account=$SERVICE_ACCOUNT_ADDRESS
```

如果要部署為 Cloud Functions 第 2 代，可以使用下列指令，透過 `trigger-event` 和 `trigger-resource` 旗標部署函式：

```
gcloud functions deploy hello-gcs-function \
  --allow-unauthenticated \
  --entry-point HelloGcs.Function \
  --gen2 \
  --region $REGION \
  --runtime dotnet8 \
  --trigger-event google.storage.object.finalize \
  --trigger-resource ${BUCKET_NAME} \
  --service-account=$SERVICE_ACCOUNT_ADDRESS
```

幾分鐘後，Cloud Console 中就會顯示函式：

Cloud Functions

Functions

CREATE FUNCTION

REFRESH

Filter

Filter functions

☐		Environment	Name ↑	Last deployed	Region	Trigger	Runtime
☐		2nd gen	hello-gcs-function	Sep 22, 2022, 3:00:57 PM	us-central1	Bucket: cloud-functions-bucket-cloud-functions-atame!	.NET 6.0
☐		2nd gen	hello-http-function	Sep 22, 2022, 2:37:58 PM	us-central1	HTTP	.NET 6.0

將檔案上傳至儲存空間 bucket，觸發函式：

```
echo "Hello from Storage" > random.txt
gsutil cp random.txt gs://${BUCKET_NAME}
```

讀取記錄，確認函式是否已觸發：

如果是 Cloud Run 函式，可以執行下列指令：

```
gcloud logging read "resource.labels.service_name=hello-gcs-function AND textPayload:
Name" --format=json
```

如果是第 2 代函式，可以執行下列指令：

```
gcloud functions logs read hello-gcs-function \
  --gen2 \
  --region us-central1
```

步驟 7 · 約 4 分鐘

7. CloudEvent 函式 - Pub/Sub

您將建立及部署 CloudEvent 函式，以回應 Cloud Pub/Sub 事件。

首先，請建立會發出事件的 Cloud Pub/Sub 主題：

```
TOPIC_NAME=cloud-functions-topic
gcloud pubsub topics create ${TOPIC_NAME}
```

使用 `gcf-event` 範本建立 CloudEvent 函式：

```
cd ..
mkdir HelloPubSub
cd HelloPubSub
dotnet new gcf-event
```

這會建立專案和 `Function.cs` 檔案，以回應 `CloudEvent` 要求。此外，系統預設會將 `CloudEvent` 的資料剖析為 `StorageObjectData`。

在 `.csproj` 檔案中，將目標架構變更為 `net8.0`：

```
<TargetFramework>net8.0</TargetFramework>
```

將 `StorageObjectData` 變更為 `MessagePublishedData`，即可剖析 Pub/Sub 訊息。將

`Google.Events.Protobuf.Cloud.Storage.V1` 變更為 `Google.Events.Protobuf.Cloud.PubSub.V1`。

最後，函式應如下所示：

```

using CloudNative.CloudEvents;
using Google.Cloud.Functions.Framework;
using Google.Events.Protobuf.Cloud.PubSub.V1;
using System;
using System.Threading;
using System.Threading.Tasks;

namespace HelloPubSub;

public class Function : ICloudEventFunction<MessagePublishedData>
{
    public Task HandleAsync(CloudEvent cloudEvent, MessagePublishedData data,
        CancellationToken cancellationToken)
    {
        var nameFromMessage = data.Message?.TextData;
        var name = string.IsNullOrEmpty(nameFromMessage) ? "world" : nameFromMessage;
        Console.WriteLine($"Hello {name}");
        return Task.CompletedTask;
    }
}

```

如要將 Cloud Run 函式直接部署至 Cloud Run，請先部署函式，然後為函式建立觸發條件。

```

gcloud beta run deploy hello-pubsub-function \
    --source . \
    --function HelloPubSub.Function \
    --region $REGION \
    --base-image dotnet8 \
    --no-allow-unauthenticated \
    --service-account=$SERVICE_ACCOUNT_ADDRESS

```

現在為 Cloud Run 函式建立觸發條件

```

gcloud eventarc triggers create my-pubsub-trigger \
    --location=$REGION \
    --service-account=$SERVICE_ACCOUNT_ADDRESS \
    --destination-run-service=hello-pubsub-function \
    --destination-run-region=$REGION \
    --destination-run-path="/" \
    --event-filters="type=google.cloud.pubsub.topic.v1.messagePublished" \
    --transport-topic=projects/$PROJECT_ID/topics/$TOPIC_NAME

```

如要部署為 Cloud Functions 第 2 代，可以使用下列指令，透過 `trigger-topic` 旗標部署函式：

```
gcloud functions deploy hello-pubsub-function \
  --allow-unauthenticated \
  --entry-point HelloPubSub.Function \
  --gen2 \
  --region us-central1 \
  --runtime dotnet8 \
  --trigger-topic ${TOPIC_NAME}
```

幾分鐘後，Cloud Console 中就會顯示函式：

Cloud Functions

Functions

CREATE FUNCTION

REFRESH

Filter

Filter functions

☐		Environment	Name	Last deployed	Region	Trigger	Runtime
☐		2nd gen	hello-gcs-function	Sep 22, 2022, 3:00:57 PM	us-central1	Bucket: cloud-functions-bucket-cloud-functions-atamei	.NET 6.0
☐		2nd gen	hello-http-function	Sep 22, 2022, 2:37:58 PM	us-central1	HTTP	.NET 6.0
☐		2nd gen	hello-pubsub-function	Sep 22, 2022, 3:14:18 PM	us-central1	Topic: cloud-functions-topic	.NET 6.0

將訊息發布至主題，藉此觸發函式：

```
gcloud pubsub topics publish ${TOPIC_NAME} --message="World"
```

讀取記錄，確認函式是否已觸發。

如果是 Cloud Run 函式，可以執行下列指令：

```
gcloud logging read "resource.labels.service_name=hello-pubsub-function AND
textPayload: World" --format=json
```

如果是第 2 代函式，可以執行下列指令：

```
gcloud functions logs read hello-pubsub-function \
  --gen2 \
  --region us-central1
```

步驟 8 · 約 4 分鐘

8. CloudEvent 函式 - 無型別

如果您正在試用 CloudEvents，還沒有要使用的酬載資料模型，或是希望函式能夠處理任何 CloudEvent，可以使用未輸入型別的 CloudEvent 函式。

使用 `gcf-untyped-event` 範本建立 CloudEvent 函式：

```
cd ..  
mkdir HelloUntyped  
cd HelloUntyped  
dotnet new gcf-untyped-event
```

這會建立專案和 `Function.cs` 檔案，回應 `CloudEvent` 要求，但不會嘗試剖析 `CloudEvent` 的資料。

在 `.csproj` 檔案中，將目標架構變更為 `net8.0`：

```
<TargetFramework>net8.0</TargetFramework>
```

如要將 Cloud Run 函式直接部署至 Cloud Run，請先部署函式，然後為函式建立觸發條件。

```
gcloud beta run deploy hello-untyped-function \  
  --source . \  
  --function HelloUntyped.Function \  
  --region $REGION \  
  --base-image dotnet8 \  
  --no-allow-unauthenticated
```

現在為 Cloud Run 函式建立觸發條件

```
BUCKET_REGION=$REGION  
  
gcloud eventarc triggers create hello-untyped-function-trigger \  
  --location=$REGION \  
  --destination-run-service=hello-untyped-function \  
  --destination-run-region=$BUCKET_REGION \  
  --event-filters="type=google.cloud.storage.object.v1.finalized" \  
  --event-filters="bucket=$BUCKET_NAME" \  
  --service-account=$SERVICE_ACCOUNT_ADDRESS
```

如要部署為 Cloud Functions 第 2 代，可以使用下列指令，透過 `trigger-event` 和 `trigger-resource` 旗標部署函式：

```
gcloud functions deploy hello-untyped-function \
  --allow-unauthenticated \
  --entry-point HelloUntyped.Function \
  --gen2 \
  --region us-central1 \
  --runtime dotnet8 \
  --trigger-event google.storage.object.finalize \
  --trigger-resource ${BUCKET_NAME}
```

只要檔案上傳至儲存空間 bucket，就會觸發函式。

幾分鐘後，Cloud Console 中就會顯示函式：

⋮

Cloud Functions

Functions

+

CREATE FUNCTION

↻

REFRESH

☰

Filter

Filter functions

☐	☒	Environment	Name ↑	Last deployed	Region	Trigger	Runtime
☐	☒	2nd gen	hello-gcs-function	Sep 22, 2022, 3:00:57 PM	us-central1	Bucket: cloud-functions-bucket-cloud-functions-atamel	.NET 6.0
☐	☒	2nd gen	hello-http-function	Sep 22, 2022, 2:37:58 PM	us-central1	HTTP	.NET 6.0
☐	☒	2nd gen	hello-pubsub-function	Sep 22, 2022, 3:14:18 PM	us-central1	Topic: cloud-functions-topic	.NET 6.0
☐	☒	2nd gen	hello-untyped-function	Sep 22, 2022, 3:22:58 PM	us-central1	Bucket: cloud-functions-bucket-cloud-functions-atamel	.NET 6.0

將檔案上傳至儲存空間 bucket，觸發函式：

```
echo "Hello from Storage" > random.txt
gsutil cp random.txt gs://${BUCKET_NAME}
```

讀取記錄，確認函式是否已觸發。

如果是 Cloud Run 函式，可以執行下列指令：

```
gcloud logging read "resource.labels.service_name=hello-gcs-function AND textPayload:
Name" --format=json
```

如果是第 2 代函式，可以執行下列指令：

```
gcloud functions logs read hello-untyped-function \
  --gen2 \
  --region us-central1
```

步驟 9 · 約 1 分鐘

9. 恭喜！

恭喜您完成本程式碼研究室。

涵蓋內容

- 適用於 .NET 的 Functions Framework。
 - 如何編寫 HTTP Cloud 函式。
 - 如何編寫 CloudEvent 函式來回應 Cloud Storage 事件。
 - 如何編寫 CloudEvent 函式來回應 Cloud Pub/Sub 事件。
 - 如何編寫 CloudEvent 函式，回應任何類型的事件。
-